

## DATA 260 – Homework 5 Report

**Name:** Shashank Ranjan **Student ID:** 018322837

**Repository:** <https://github.com/ShankySJSU/Data260-2837>

**Submission tag:** hw5

**Tagged commit hash:** 5e232b172288e4e7e70255ad90ffd221caa6d55a

**Collaborators with access:** Sbnikitha, supriyaselvangesan

### Section 0 – Personal Configuration and Domain

| Value           | Definition                                    | Result                       |
|-----------------|-----------------------------------------------|------------------------------|
| SID4            | Last four digits of SJSU ID                   | 2837                         |
| PORT_BASE       | $8000 + (\text{SID4} \bmod 900) = 8000 + 137$ | 8137                         |
| PREFIX          | "s" + SID4                                    | s2837                        |
| SEED            | SID4                                          | 2837                         |
| VERIFY_SEED     | $260000 + \text{SID4}$                        | 262837                       |
| DOMAIN_ID       | $\text{SID4} \bmod 8$                         | 5                            |
| Assigned domain | Determined by DOMAIN_ID                       | Local restaurant inspections |

**Hardware:** Local Windows PC, Intel i7 CPU, 16 GB RAM, Python 3.12, VS Code.

**Models:** Local Ollama (qwen2.5:1.5b, qwen3:4b, llama3.2:1b); embeddings: sentence-transformers/all-MiniLM-L6-v2.

MODEL = "qwen2.5:1.5b" was used to produce the Part 5 Scenario results

Additional NOTE: All my output shows my directory information where I have executed my code. You will notice my last 4 digit of student id (2837) be part of my directory. Hence rather than showing my name (Shshank Ranjan), this confirms that I have executed all the code myself and in my computer.

```
C:\SJSURepo\Data260\Data260-2837\code\web_application>python verify_hw05.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
PASS unknown_tool
PASS safety_rule_allows_valid_score
PASS safety_rule_blocks_invalid_score
PASS agent_stops_at_max_steps

10/10 tests passed

HW5 verification completed.
Output written to: C:\SJSURepo\Data260\Data260-2837\reports\hw05\verification.json
10/10 checks passed

C:\SJSURepo\Data260\Data260-2837\code\web_application>
```

*Handwritten notes:*

- A red box highlights the directory path `Data260-2837` in the command prompt, with a red arrow pointing to it.
- Handwritten text "Student Id" is written in red, with a purple box around the student ID "018322837".
- A red arrow points from the student ID box down to the directory path in the command prompt.

## Base Directory Structure:

The image shows the VS Code Explorer interface for a project named 'Data260-2837'. The Explorer is divided into three panes. The left pane shows the overall project structure, with 'web\_application' selected. The middle pane shows the contents of 'web\_application', including 'hw2-shashank.html', 'hw3\_auth.py', 'hw3\_main.py', 'hw4\_database.py', 'hw4\_main.py', 'hw5\_main.py', 'migrate\_hw5.py', 'restfeedback.js', 'verify\_hw02.py', 'verify\_hw04.py', 'verify\_hw05.py', and '\_\_init\_\_.py'. The right pane shows the contents of the 'mcp' folder, which is highlighted with a red box. The 'mcp' folder contains the following files: '\_\_pycache\_\_', 'agent.py', 'domain\_server.py', 'execute\_tool.py', 'fault\_injection\_runner.py', 'meals\_server.py', 'retry\_utils.py', 'run\_agent\_scenarios.py', and 'test\_execute\_tool\_offline.py'.

```
graph TD
    Data260-2837 --> code
    code --> web_application
    web_application --> __pycache__
    web_application --> domain
    web_application --> mcp
    web_application --> templates
    web_application --> tests
    web_application --> __init__.py
    web_application --> auth.py
    web_application --> database.py
    web_application --> hw1_Dockerfile
    web_application --> hw2_agents_demo.py
    web_application --> hw2_compare_ceiling.py
    web_application --> hw2_graph.py
```

**Left Pane (Explorer):**

- ▼ Data260-2837
  - ▼ code
    - > \_\_pycache\_\_
    - > rag
    - > react\_client
    - ▼ web\_application
      - > \_\_pycache\_\_
      - > domain
      - > mcp
      - > templates
      - > tests
      - 🔗 \_\_init\_\_.py
      - 🔗 auth.py
      - 🔗 database.py
      - 📄 hw1\_Dockerfile
      - 🔗 hw2\_agents\_demo.py
      - 🔗 hw2\_compare\_ceiling.py
      - 🔗 hw2\_graph.py

**Middle Pane (Data260-2837/code/web\_application):**

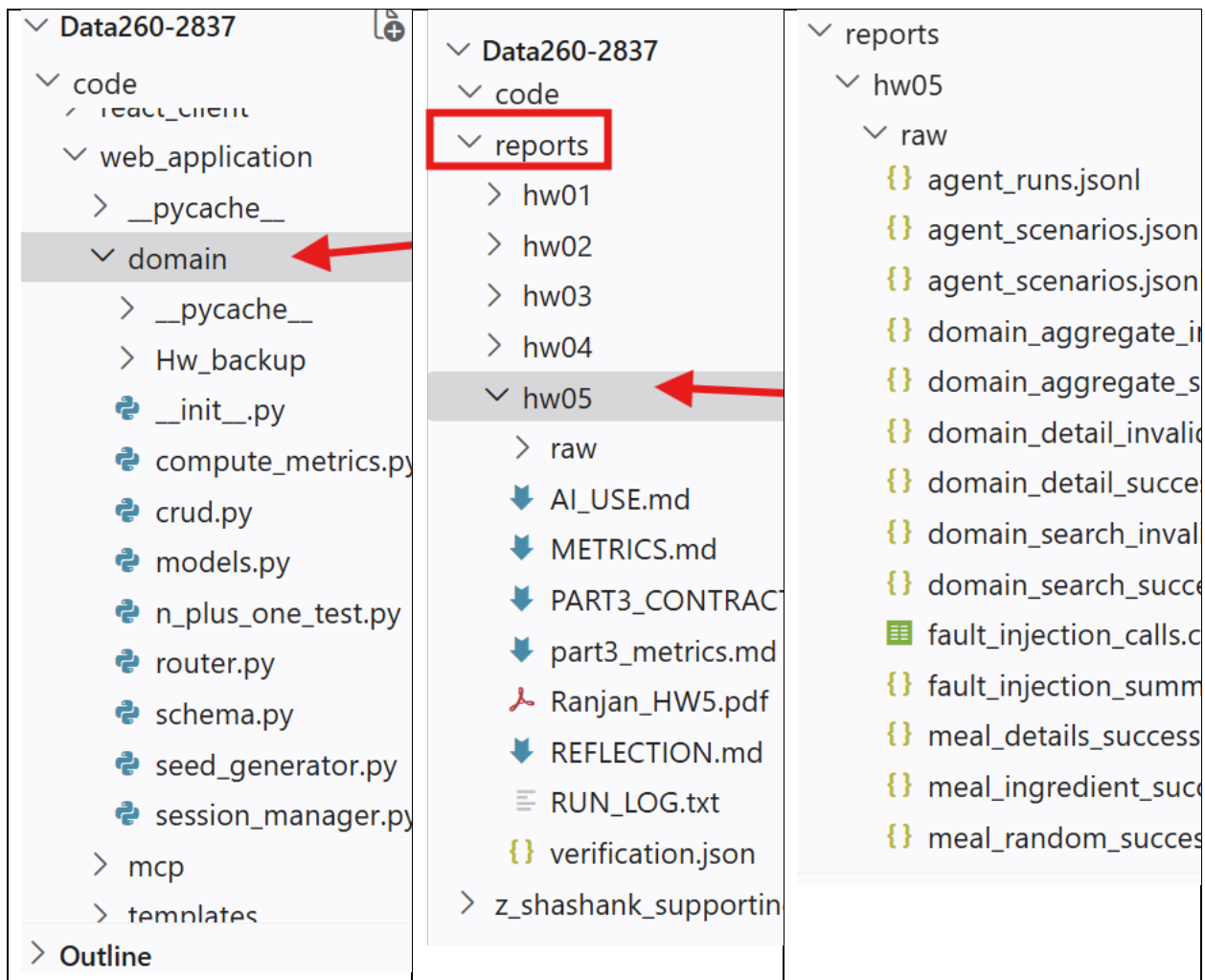
- ▼ Data260-2837
  - ▼ code
    - ▼ web\_application
      - 🔗 hw2-shashank.html
      - 🔗 hw3\_auth.py
      - 🔗 hw3\_main.py
      - 🔗 hw4\_database.py
      - 🔗 hw4\_main.py
      - 🔗 hw5\_main.py
      - 🔗 migrate\_hw5.py
      - 📄 restfeedback.js
      - 🔗 verify\_hw02.py
      - 🔗 verify\_hw04.py
      - 🔗 verify\_hw05.py
      - 🔗 \_\_init\_\_.py
      - > reports
      - > z\_shashank\_supportingfiles

**Right Pane (Data260-2837/code/web\_application/mcp):**

- ▼ Data260-2837
  - ▼ code
    - ▼ web\_application
      - > \_\_pycache\_\_
      - > domain
      - ▼ mcp
        - > \_\_pycache\_\_
        - 🔗 agent.py
        - 🔗 domain\_server.py
        - 🔗 execute\_tool.py
        - 🔗 fault\_injection\_runner.py
        - 🔗 meals\_server.py
        - 🔗 retry\_utils.py
        - 🔗 run\_agent\_scenarios.py
        - 🔗 test\_execute\_tool\_offline.py
      - > templates
      - 🔗 \_\_init\_\_.py
      - 🔗 auth.py

**Bottom Bar:**

- > Outline
- > Timeline



## Part 1 – Database, API, and Redux Client

### I. Database

The HW4 database was migrated in place: migrate\_hw5.py added the inspection\_code, inspection\_date, score, restaurant\_id, created\_at and updated\_at columns, created a legacy restaurant record for existing rows, added a unique index on inspection\_code and a foreign key to restaurant.

Screenshot 1.1 – Migration run (terminal)

```
C:\SJSURepo\Data260\Data260-2837\code\web_application>python migrate_hw5
.py
Starting HW5 database migration...
Adding column: inspection_code
Adding column: inspection_date
Adding column: score
Adding column: restaurant_id
Adding column: created_at
Adding column: updated_at
Creating legacy restaurant record...
C:\SJSURepo\Data260\Data260-2837\code\web_application\migrate_hw5.py:151
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and sched
uled for removal in a future version. Use timezone-aware objects to repr
esent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    "created_at": datetime.utcnow(),
C:\SJSURepo\Data260\Data260-2837\code\web_application\migrate_hw5.py:152
: DeprecationWarning: datetime.datetime.utcnow() is deprecated and sched
uled for removal in a future version. Use timezone-aware objects to repr
esent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    "updated_at": datetime.utcnow(),
Adding unique index for inspection_code...
Adding restaurant foreign key...
HW5 database migration completed successfully.

C:\SJSURepo\Data260\Data260-2837\code\web_application>
```

Screenshot 1.2 – MySQL: restaurant and restaurant\_inspection tables with restaurant\_id foreign key

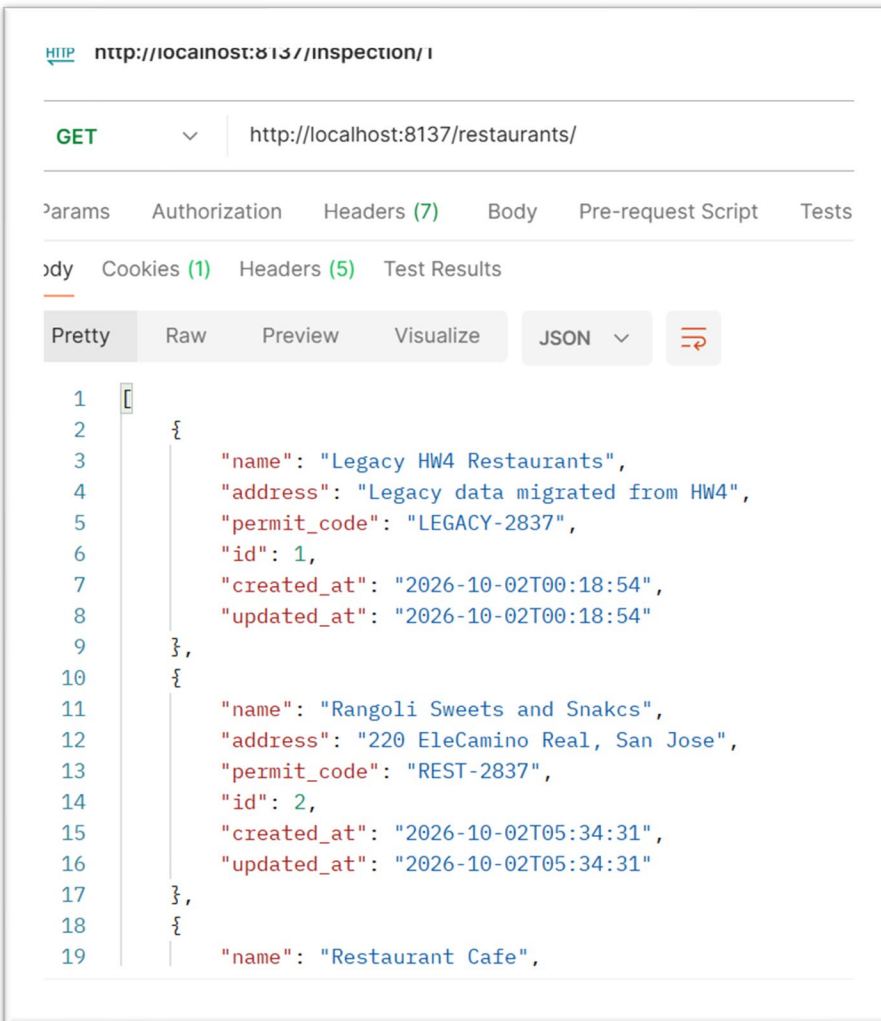
```
4 • SELECT COUNT(*) FROM restaurant_inspection;
5
6 • select * from restaurant_inspection;
7
8 • select * from restaurant;
9
10 • SELECT id, name, inspection_code, score, restaurant_id
11 FROM restaurant_inspection
12 # where restaurant_id = 2
13 LIMIT 5;
```

| sult Grid   Filter Rows:   Edit:   Export/Import:   Wrap C |                                            |                 |       |               |
|------------------------------------------------------------|--------------------------------------------|-----------------|-------|---------------|
| id                                                         | name                                       | inspection_code | score | restaurant_id |
| 1                                                          | Rangoli Sweets and Snacks                  | INSP-00000001   | 0     | 1             |
| 2                                                          | Moonlight Garden Restaurant & Banquet Hall | INSP-00000002   | 60    | 1             |
| 4                                                          | Restaurant_Bar_0                           | INSP-00000004   | 0     | 11            |
| 5                                                          | Restaurant_Garden_1                        | INSP-00000005   | 55    | 12            |
| 6                                                          | Restaurant_Cafe_2                          | INSP-00000006   | 0     | 3             |
| NULL                                                       | NULL                                       | NULL            | NULL  | NULL          |

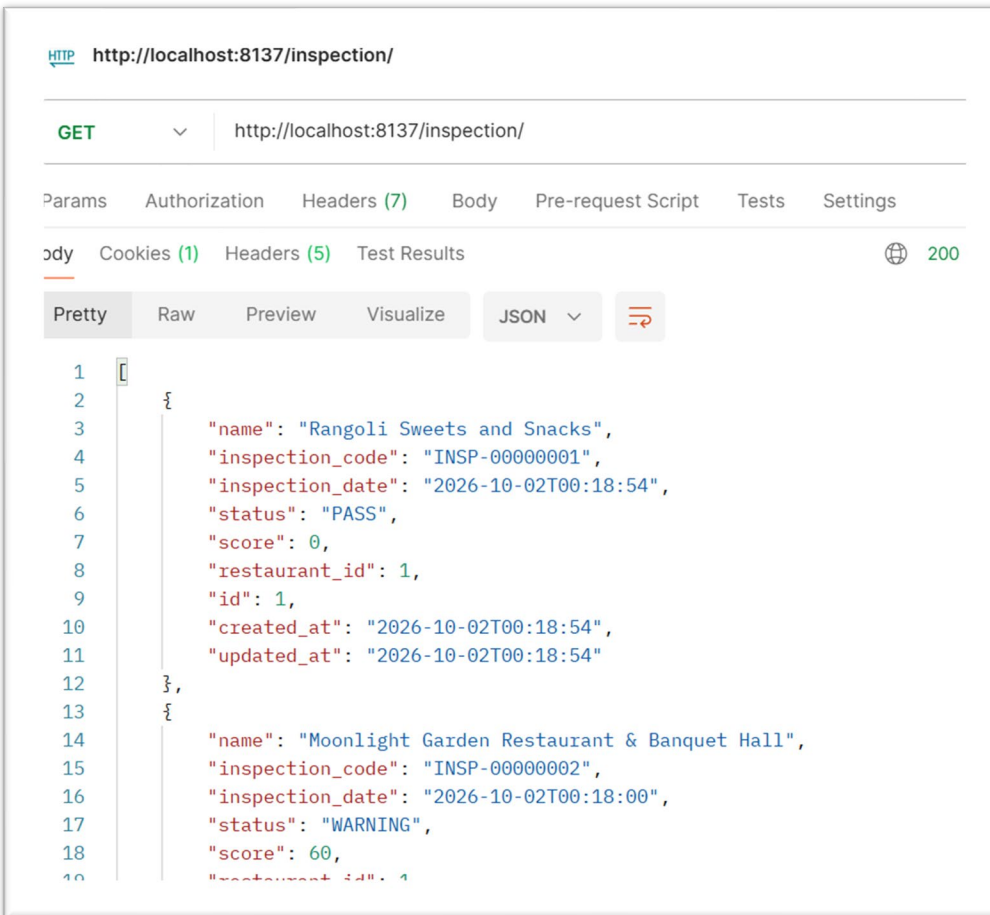
## II. API Implementation (Postman)

Verified inspection end-points: using POST MAN

- List of restaurants using GET (<http://localhost:8137/restaurants/>)



- List of Inspection using Get (<http://localhost:8137/inspection/>)



- Created Restaurant using **POST** and JSON. (raw body) ,  
<http://localhost:8137/restaurants/>

```
{
  "name": "Restaurant_Fry_Chicknet_99",
  "address": "Sunnyvale",
  "permit_code": "PERM099",
  "id": 13
}
```

Body Cookies (1) Headers (5) Test Results 201 Created 42 ms 306 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "name": "Restaurant_Fry_Chicknet_99",
3   "address": "Sunnyvale",
4   "permit_code": "PERM099",
5   "id": 13,
6   "created_at": "2026-10-04T05:54:00",
7   "updated_at": "2026-10-04T05:54:00"
8 }
```

GET http://localhost:8037/inspection/1

Params Auth Headers (7) Body Pre-req. Tests Settings

Body Cookies (1) Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "name": "Rangoli Sweets and Snacks",
3   "inspection_code": "INSP-00000001",
4   "inspection_date": "2026-10-02T00:18:54",
5   "status": "PASS",
6   "score": 0,
7   "restaurant_id": 1,
8   "id": 1,
9   "created_at": "2026-10-02T00:18:54",
10  "updated_at": "2026-10-02T00:18:54"
11 }
```

Updating the ID= 1 from score = 0 to 90

The screenshot shows a REST client interface with a PUT request to `http://localhost:8137/inspection/1`. The request body is a JSON object with the following fields: `name`, `inspection_code`, `inspection_date`, `status`, `score`, `restaurant_id`, and `id`. The `score` field is highlighted with a red box and a red arrow pointing to the `Test Results` tab, indicating the update. The `JSON` format is selected in the dropdown menu.

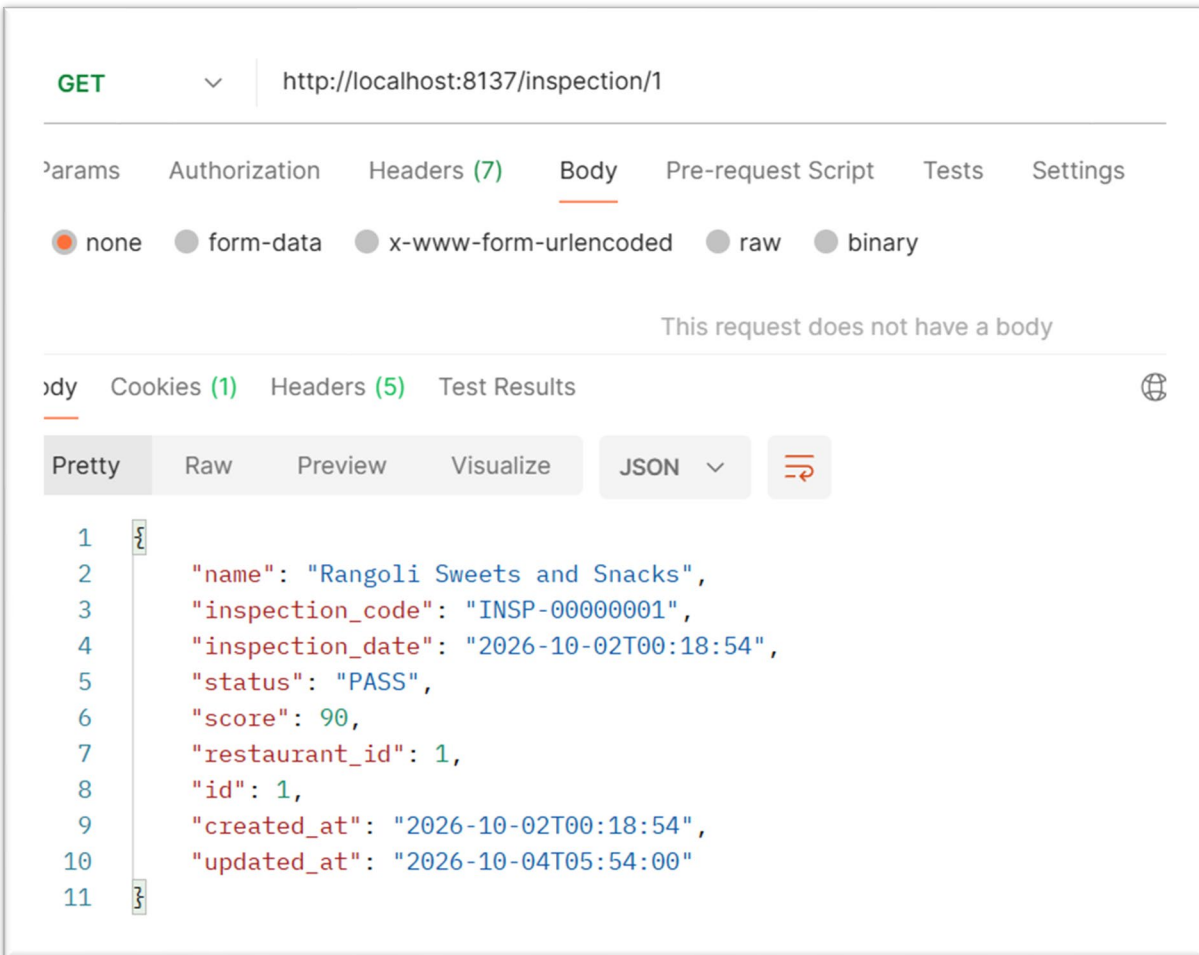
**Request Body (JSON):**

```
1 {
2   "name": "Rangoli Sweets and Snacks",
3   "inspection_code": "INSP-00000001",
4   "inspection_date": "2026-10-02T00:18:54",
5   "status": "PASS",
6   "score": 90,
7   "restaurant_id": 1,
8   "id": 1
}
```

**Response (JSON):**

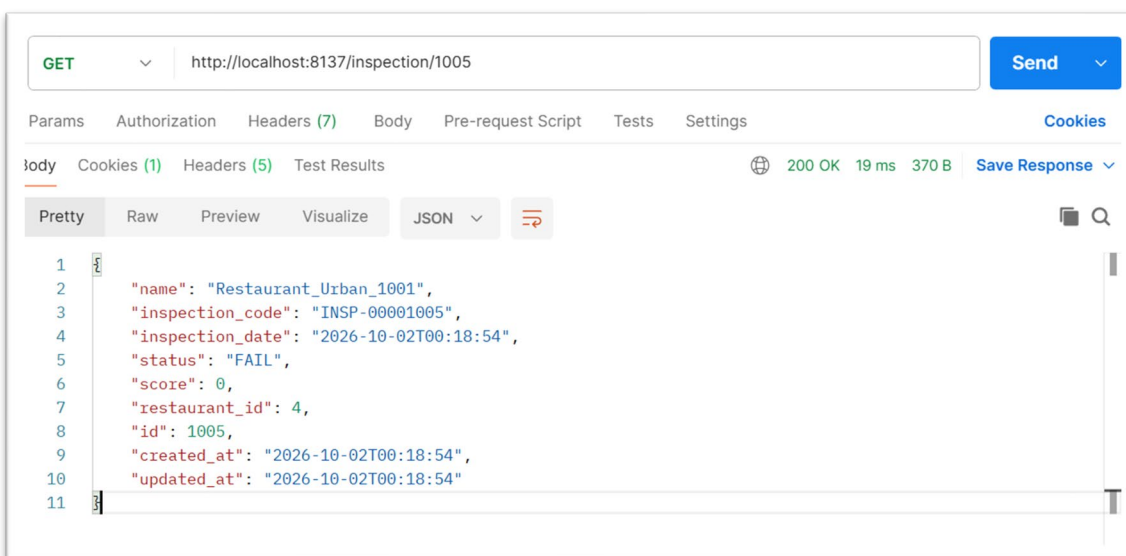
```
1 {
2   "name": "Rangoli Sweets and Snacks",
3   "inspection_code": "INSP-00000001",
4   "inspection_date": "2026-10-02T00:18:54",
5   "status": "PASS",
6   "score": 90,
7   "restaurant_id": 1,
8   "id": 1,
}
```

The response status is `200 OK` with a response time of `43 ms` and a size of `372 B`. The `Save Response` button is visible.

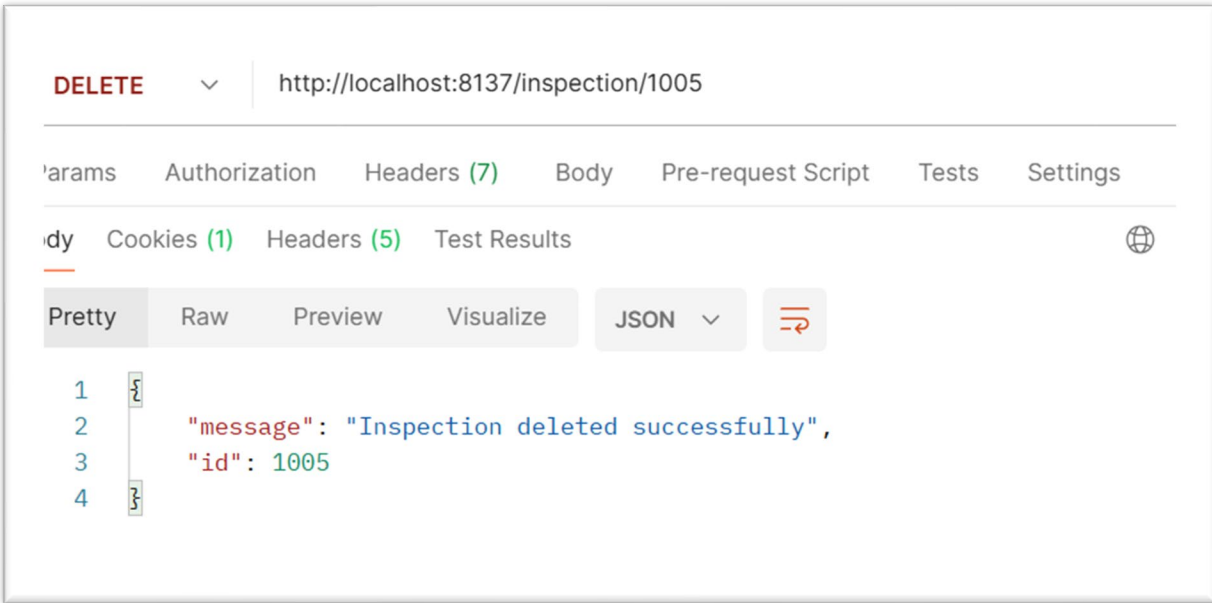


Going to delete restaurant inspection with ID = 1005.

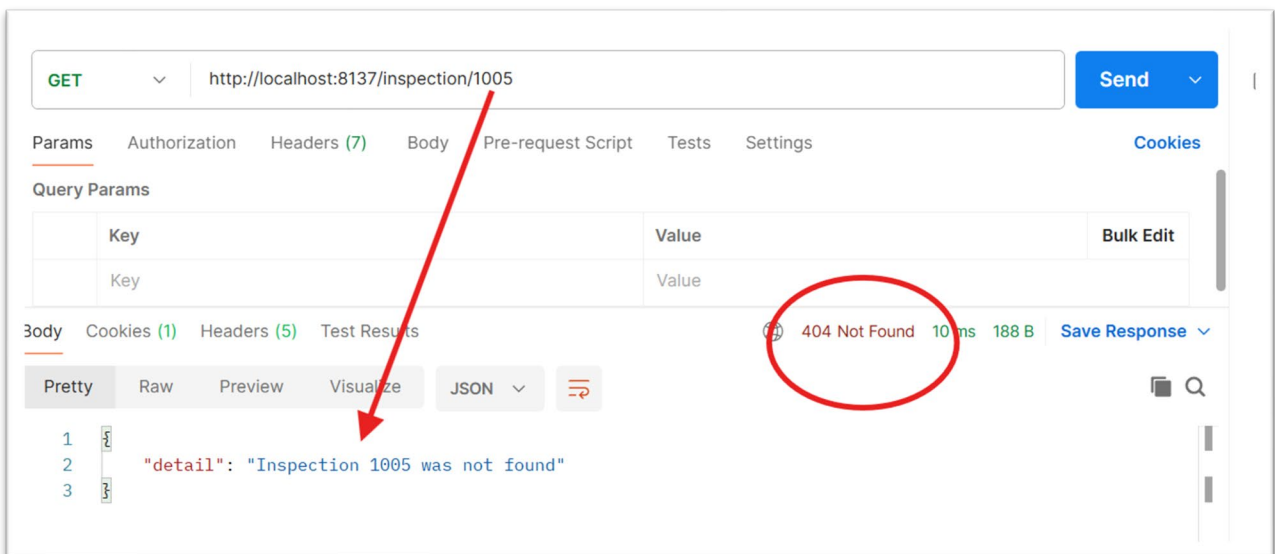
Step 1: Verify that restaurant inspection with ID = 1005 exists:

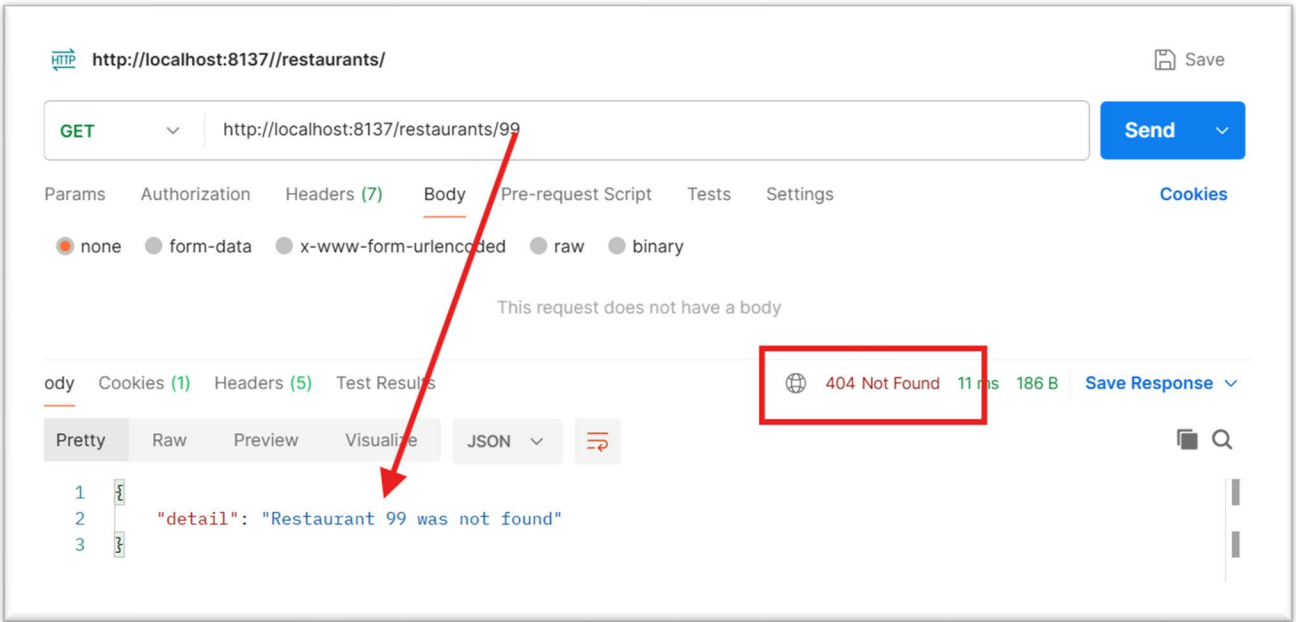


Step 2: Deleting the inspection with ID = 1005

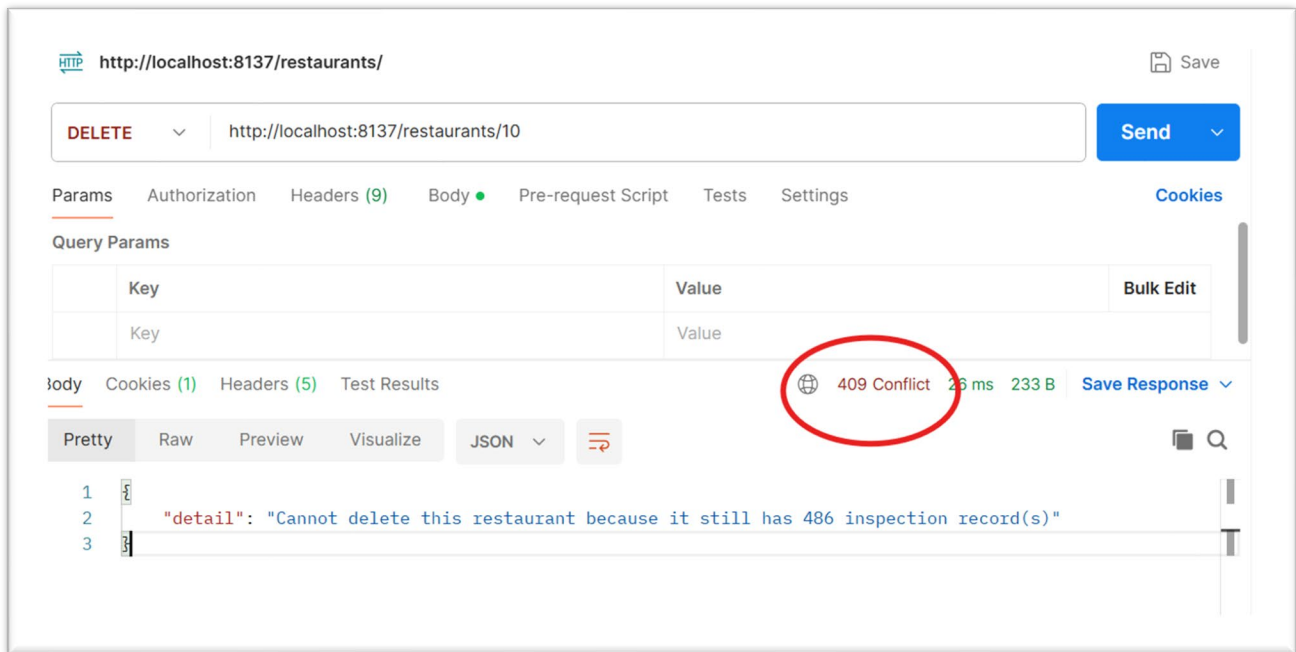


Step 3: Verifying that Inspection ID = 1005 does not exists (and also validating the negative test) Showing Error 404 (Not found)





Constraint and conflict (when trying to delete restaurant when it has number of inspection records)



### III. Redux Toolkit Client

Redux store, an inspections slice, and axios async thunks (fetch, create, update, delete) call the FastAPI endpoints on port 8137.

### Screenshot 1.3 – Home screen: list read from Redux state

**Restaurant Inspections**

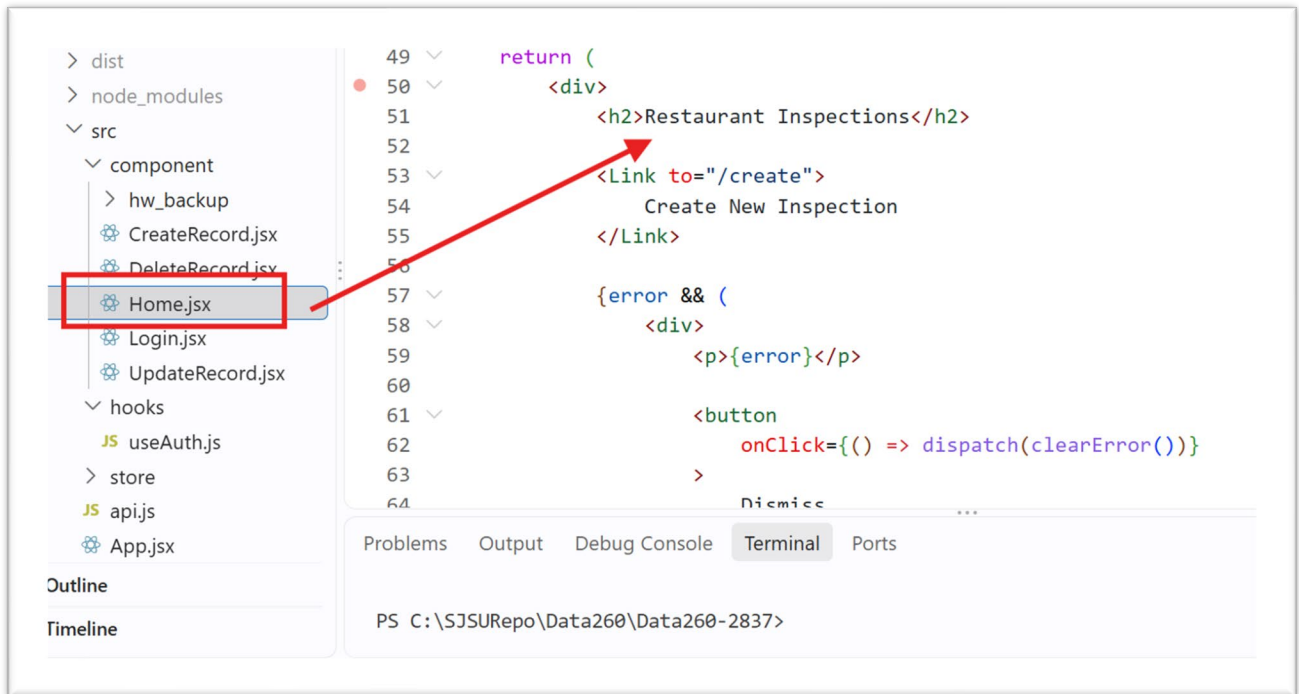
[Create New Inspection](#)

- Rangoli Sweets and Snacks | Code: INSP-00000001 | Status: PASS | Score: 0 | Restaurant ID: 1 [Update](#) [Delete](#)
- Moonlight Garden Restaurant & Banquet Hall | Code: INSP-00000002 | Status: WARNING | Score: 60 | Restaurant ID: 1 [Update](#) [Delete](#)
- Restaurant\_Bar\_0 | Code: INSP-00000004 | Status: PASS | Score: 0 | Restaurant ID: 11 [Update](#) [Delete](#)
- Restaurant\_Garden\_1 | Code: INSP-00000005 | Status: FAIL | Score: 0 | Restaurant ID: 12 [Update](#) [Delete](#)
- Restaurant\_Cafe\_2 | Code: INSP-00000006 | Status: FAIL | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Cafe\_3 | Code: INSP-00000007 | Status: PASS | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Urban\_4 | Code: INSP-00000008 | Status: PASS | Score: 0 | Restaurant ID: 4 [Update](#) [Delete](#)
- Restaurant\_Golden\_5 | Code: INSP-00000009 | Status: PASS | Score: 0 | Restaurant ID: 5 [Update](#) [Delete](#)
- Restaurant\_Golden\_6 | Code: INSP-00000010 | Status: PASS | Score: 0 | Restaurant ID: 5 [Update](#) [Delete](#)
- Restaurant\_Grill\_7 | Code: INSP-00000011 | Status: WARNING | Score: 0 | Restaurant ID: 6 [Update](#) [Delete](#)
- Restaurant\_Bar\_8 | Code: INSP-00000012 | Status: FAIL | Score: 0 | Restaurant ID: 11 [Update](#) [Delete](#)
- Restaurant\_Bistro\_9 | Code: INSP-00000013 | Status: PASS | Score: 0 | Restaurant ID: 7 [Update](#) [Delete](#)
- Restaurant\_Spicy\_10 | Code: INSP-00000014 | Status: WARNING | Score: 0 | Restaurant ID: 8 [Update](#) [Delete](#)
- Restaurant\_Rustic\_11 | Code: INSP-00000015 | Status: WARNING | Score: 0 | Restaurant ID: 9 [Update](#) [Delete](#)
- Restaurant\_Grill\_12 | Code: INSP-00000016 | Status: FAIL | Score: 0 | Restaurant ID: 6 [Update](#) [Delete](#)
- Restaurant\_Rustic\_13 | Code: INSP-00000017 | Status: FAIL | Score: 0 | Restaurant ID: 9 [Update](#) [Delete](#)
- Restaurant\_Grill\_14 | Code: INSP-00000018 | Status: WARNING | Score: 0 | Restaurant ID: 6 [Update](#) [Delete](#)
- Restaurant\_Deli\_15 | Code: INSP-00000019 | Status: PASS | Score: 0 | Restaurant ID: 10 [Update](#) [Delete](#)
- Restaurant\_Cafe\_16 | Code: INSP-00000020 | Status: PASS | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Deli\_17 | Code: INSP-00000021 | Status: FAIL | Score: 0 | Restaurant ID: 10 [Update](#) [Delete](#)
- Restaurant\_Spicy\_18 | Code: INSP-00000022 | Status: FAIL | Score: 0 | Restaurant ID: 8 [Update](#) [Delete](#)
- Restaurant\_Bar\_19 | Code: INSP-00000023 | Status: WARNING | Score: 0 | Restaurant ID: 11 [Update](#) [Delete](#)
- Restaurant\_Garden\_20 | Code: INSP-00000024 | Status: FAIL | Score: 0 | Restaurant ID: 12 [Update](#) [Delete](#)
- Restaurant\_Cafe\_21 | Code: INSP-00000025 | Status: PASS | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Spicy\_22 | Code: INSP-00000026 | Status: WARNING | Score: 0 | Restaurant ID: 8 [Update](#) [Delete](#)
- Restaurant\_Golden\_23 | Code: INSP-00000027 | Status: FAIL | Score: 0 | Restaurant ID: 5 [Update](#) [Delete](#)
- Restaurant\_Bistro\_24 | Code: INSP-00000028 | Status: FAIL | Score: 0 | Restaurant ID: 7 [Update](#) [Delete](#)
- Restaurant\_Bar\_25 | Code: INSP-00000029 | Status: PASS | Score: 0 | Restaurant ID: 11 [Update](#) [Delete](#)
- Restaurant\_Cafe\_26 | Code: INSP-00000030 | Status: FAIL | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Bistro\_27 | Code: INSP-00000031 | Status: FAIL | Score: 0 | Restaurant ID: 7 [Update](#) [Delete](#)
- Restaurant\_Rustic\_28 | Code: INSP-00000032 | Status: FAIL | Score: 0 | Restaurant ID: 9 [Update](#) [Delete](#)
- Restaurant\_Golden\_29 | Code: INSP-00000033 | Status: WARNING | Score: 0 | Restaurant ID: 5 [Update](#) [Delete](#)
- Restaurant\_Garden\_30 | Code: INSP-00000034 | Status: FAIL | Score: 0 | Restaurant ID: 12 [Update](#) [Delete](#)
- Restaurant\_Grill\_31 | Code: INSP-00000035 | Status: FAIL | Score: 0 | Restaurant ID: 6 [Update](#) [Delete](#)
- Restaurant\_Garden\_32 | Code: INSP-00000036 | Status: PASS | Score: 0 | Restaurant ID: 12 [Update](#) [Delete](#)
- Restaurant\_Cafe\_33 | Code: INSP-00000037 | Status: PASS | Score: 0 | Restaurant ID: 3 [Update](#) [Delete](#)
- Restaurant\_Rustic\_34 | Code: INSP-00000038 | Status: FAIL | Score: 0 | Restaurant ID: 9 [Update](#) [Delete](#)
- Restaurant\_Rustic\_35 | Code: INSP-00000039 | Status: WARNING | Score: 0 | Restaurant ID: 9 [Update](#) [Delete](#)
- Restaurant\_Spicy\_36 | Code: INSP-00000040 | Status: WARNING | Score: 0 | Restaurant ID: 8 [Update](#) [Delete](#)

### Screenshot 1.4 – Network tab: GET /inspection/?page=1&page\_size=100 triggered by the fetch thunk

The screenshot shows the Chrome DevTools Network tab with a list of network requests. The request 'inspection/?page=1&page\_size=100' is highlighted with a red box. The table below shows the details of the requests.

| Name                             | Status | Type      | Initiator                        | Size         | Time    |
|----------------------------------|--------|-----------|----------------------------------|--------------|---------|
| react-redux.js?v=729c9485        | 200    | script    | main.jsx4                        | (disk cache) | 4 ms    |
| App.jsx                          | 304    | script    | main.jsx5                        | 0.2 kB       | 5 ms    |
| store.js                         | 304    | script    | main.jsx6                        | 0.2 kB       | 6 ms    |
| env.mjs                          | 304    | script    | client:1                         | 0.2 kB       | 6 ms    |
| chunk-RLJ2RCIQ.js?v=729c9485     | 200    | script    | react.js?v=729c9485:3            | (disk cache) | 4 ms    |
| chunk-DC5AMYBS.js?v=729c9485     | 200    | script    | react.js?v=729c9485:4            | (disk cache) | 3 ms    |
| chunk-NUMECKU6.js?v=729c9485     | 200    | script    | react-dom.client.js?v=729c9485:3 | (disk cache) | 18 ms   |
| react-router-dom.js?v=729c9485   | 200    | script    | App.jsx1                         | (disk cache) | 8 ms    |
| Login.jsx                        | 304    | script    | App.jsx3                         | 0.2 kB       | 9 ms    |
| Home.jsx                         | 304    | script    | App.jsx4                         | 0.2 kB       | 9 ms    |
| CreateRecord.jsx                 | 304    | script    | App.jsx5                         | 0.2 kB       | 9 ms    |
| UpdateRecord.jsx                 | 304    | script    | App.jsx6                         | 0.2 kB       | 11 ms   |
| DeleteRecord.jsx                 | 304    | script    | App.jsx7                         | 0.2 kB       | 11 ms   |
| @reduxjs_toolkit.js?v=729c9485   | 200    | script    | store.js1                        | (disk cache) | 9 ms    |
| inspectionSlice.js               | 304    | script    | store.js2                        | 0.2 kB       | 10 ms   |
| *?token=G45SXCOU6sAK             | 101    | websocket | client:536                       | 0.0 kB       | Pending |
| api.js                           | 304    | script    | Login.jsx2                       | 0.2 kB       | 2 ms    |
| useAuth.js                       | 304    | script    | Home.jsx5                        | 0.2 kB       | 2 ms    |
| axios.js?v=729c9485              | 200    | script    | api.js:1                         | (disk cache) | 1 ms    |
| me                               | 200    | xhr       | useAuth.js8                      | 0.3 kB       | 8 ms    |
| me                               | 200    | xhr       | useAuth.js8                      | 0.3 kB       | 33 ms   |
| inspection/?page=1&page_size=100 | 200    | xhr       | inspectionSlice.js8              | 23.0 kB      | 22 ms   |



```
import { createAsyncThunk, createSlice } from "@reduxjs/toolkit";
import api from "../api";
```

```
export const fetchInspections = createAsyncThunk(
  "inspections/fetchInspections",
  async (_, { rejectWithValue }) => {
    try {
      const response = await api.get("/inspection/", {
        params: {
          page: 1,
          page_size: 100,
        },
      });
    } catch (error) {
      return rejectWithValue(
        error.response?.data?.detail ||
        "Unable to load inspections"
      );
    }
  }
);
```

Screenshot 1.5 – Create screen

### Create Inspection Record

Restaurant Name:

Inspection Code:

Inspection Date:

Status:

Score:

Restaurant ID:

Screenshot 1.6 – Backend log for the create thunk

```
INFO: 127.0.0.1:58708 - "POST /inspection/ HTTP/1.1" 201 Created
INFO: 127.0.0.1:58708 - "GET /me HTTP/1.1" 200 OK
INFO: 127.0.0.1:58708 - "GET /inspection/?page=1&page_size=100 HTTP/1.1" 200 OK
```

```
export const createInspection = createAsyncThunk(
  "inspections/createInspection",
  async (inspection, { rejectWithValue }) => {
    try {
      const response = await api.post(
        "/inspection/",
        inspection
      );

      return response.data;
    } catch (error) {
      return rejectWithValue(
        error.response?.data?.detail ||
        "Unable to create inspection"
      );
    }
  }
);
```

Screenshot 1.7 – Update screen (inspection 5: status Fail to Warning, score 55)

### Update Inspection Record

Restaurant Name:

Inspection Code:

Inspection Date:

Status:

Score:

Restaurant ID:

Screenshot 1.8 – Backend log showing PUT /inspection/5 returning 200

```

INFO: 127.0.0.1:64217 - "GET /inspection/?page=1&page_size=100 HTTP/1.1" 200 OK
INFO: 127.0.0.1:64217 - "GET /inspection/5 HTTP/1.1" 200 OK
INFO: 127.0.0.1:64217 - "GET /inspection/5 HTTP/1.1" 200 OK
INFO: 127.0.0.1:65352 - "OPTIONS /inspection/5 HTTP/1.1" 200 OK
INFO: 127.0.0.1:49756 - "PUT /inspection/5 HTTP/1.1" 200 OK
INFO: 127.0.0.1:49756 - "GET /me HTTP/1.1" 200 OK
INFO: 127.0.0.1:49756 - "GET /me HTTP/1.1" 200 OK
INFO: 127.0.0.1:49756 - "GET /inspection/?page=1&page_size=100 HTTP/1.1" 200 OK

```

Screenshot 1.9 – Home screen showing the updated record

|                                              |                     |                 |           |                   |                                               |
|----------------------------------------------|---------------------|-----------------|-----------|-------------------|-----------------------------------------------|
| • Rangoli Sweets and Snacks                  | Code: INSP-00000001 | Status: PASS    | Score: 0  | Restaurant ID: 1  | <a href="#">Update</a> <a href="#">Delete</a> |
| • Moonlight Garden Restaurant & Banquet Hall | Code: INSP-00000002 | Status: WARNING | Score: 60 | Restaurant ID: 1  | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Bar_0                           | Code: INSP-00000004 | Status: PASS    | Score: 0  | Restaurant ID: 11 | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Garden_1                        | Code: INSP-00000005 | Status: WARNING | Score: 55 | Restaurant ID: 12 | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Cafe_2                          | Code: INSP-00000006 | Status: FAIL    | Score: 0  | Restaurant ID: 3  | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Cafe_3                          | Code: INSP-00000007 | Status: PASS    | Score: 0  | Restaurant ID: 3  | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Urban_4                         | Code: INSP-00000008 | Status: PASS    | Score: 0  | Restaurant ID: 4  | <a href="#">Update</a> <a href="#">Delete</a> |
| • Restaurant_Golden_5                        | Code: INSP-00000009 | Status: PASS    | Score: 0  | Restaurant ID: 5  | <a href="#">Update</a> <a href="#">Delete</a> |

```

export const updateInspection = createAsyncThunk(
  "inspections/updateInspection",
  async ({ id, data }, { rejectWithValue }) => {
    try {
      const response = await api.put(
        `/inspection/${id}`,
        data
      )
    } catch (error) {
      return rejectWithValue(error.message)
    }
  }
)

```

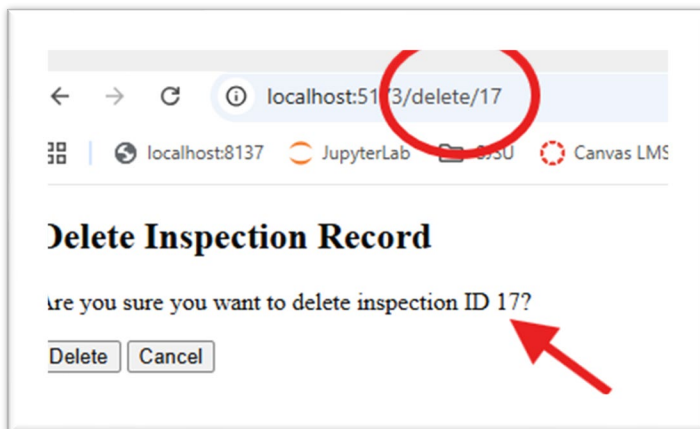
```

    );

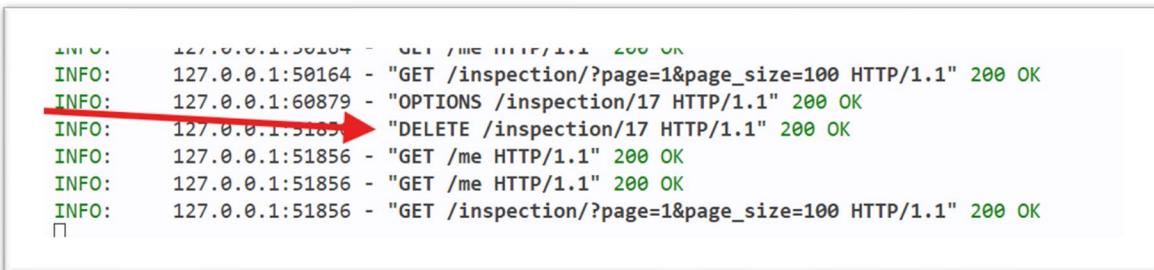
    return response.data;
  } catch (error) {
    return rejectWithValue(
      error.response?.data?.detail ||
      "Unable to update inspection"
    );
  }
}
);

```

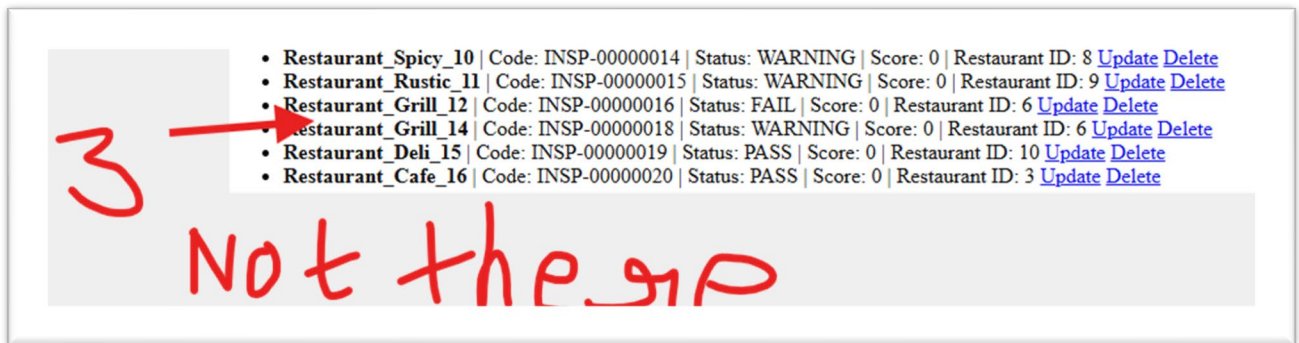
*Screenshot 1.10 – Delete confirmation for inspection 17*



*Screenshot 1.11 – Backend log showing DELETE /inspection/17 returning 200*



Screenshot 1.12 – Home screen after deletion (record 17 no longer listed)



```
export const deleteInspection = createAsyncThunk(
  "inspections/deleteInspection",
  async (id, { rejectWithValue }) => {
    try {
      await api.delete(`/inspection/${id}`);
      return id;
    } catch (error) {
      return rejectWithValue(
        error.response?.data?.detail ||
        "Unable to delete inspection"
      );
    }
  }
);
```

## Part 2 – MCP Tool Servers

### A. TheMealDB Server (meals)

Local FastMCP server (meals\_server.py) over STDIO, started with mcp dev and tested in MCP Inspector. Four tools are registered.

Screenshot 2.1 – Inspector: meals server tools list

**search\_meals\_by\_name** ▼

Search TheMealDB meals by name. Args: query: Meal name or search text.  
limit: Number of results from 1 to 25.

Edit as JSON ☐

Query \*  
chicken ↵ ✕

Limit  
5 ^ ▼

Execute Tool

**Tools**  
  
☐ Paginated  

search\_meals\_by\_name  
meals\_by\_ingredient  
random\_meal  
meal\_details

**Results**

1 {  
2 "id": "52795",  
3 "name": "Chicken Handi",  
4 "area": "India",  
5 "category": "Chicken",  
6 "thumb": "https://www.themealdb.com/images/media/meals/wywxsp148",  
7 }

1 {  
2 "id": "53358",  
3 "name": "Chicken Mandi",  
4 "area": "India",  
5 "category": "Chicken",  
6 "thumb": "https://www.themealdb.com/images/media/meals/er4d08176",  
7 }

1 {  
2 "id": "53110",  
3 "name": "Sticky Chicken",  
4 "area": "Australian",  
5 "category": "Chicken",  
6 "thumb": "https://www.themealdb.com/images/media/meals/cj56fs176",  
7 }

*Screenshot 2.2 – search\_meals\_by\_name (input and output) (I used “chicken”)*

*Screenshot 2.3 – meals\_by\_ingredient (ingredient = chicken, limit = 12)*

The screenshot displays a web application interface with a sidebar on the left and a main content area on the right.

**Tools Sidebar:**

- Tools** header
- Search tools... (input field)
- ☒ Paginated (toggle)
- search\_meals\_by\_name
- meals\_by\_ingredient** (highlighted)
- random\_meal
- meal\_details

**meals\_by\_ingredient Tool Configuration:**

**meals\_by\_ingredient**

Find meals by main ingredient. Args: ingredient: Main ingredient, such as chicken. limit: Number of results from 1 to 25.

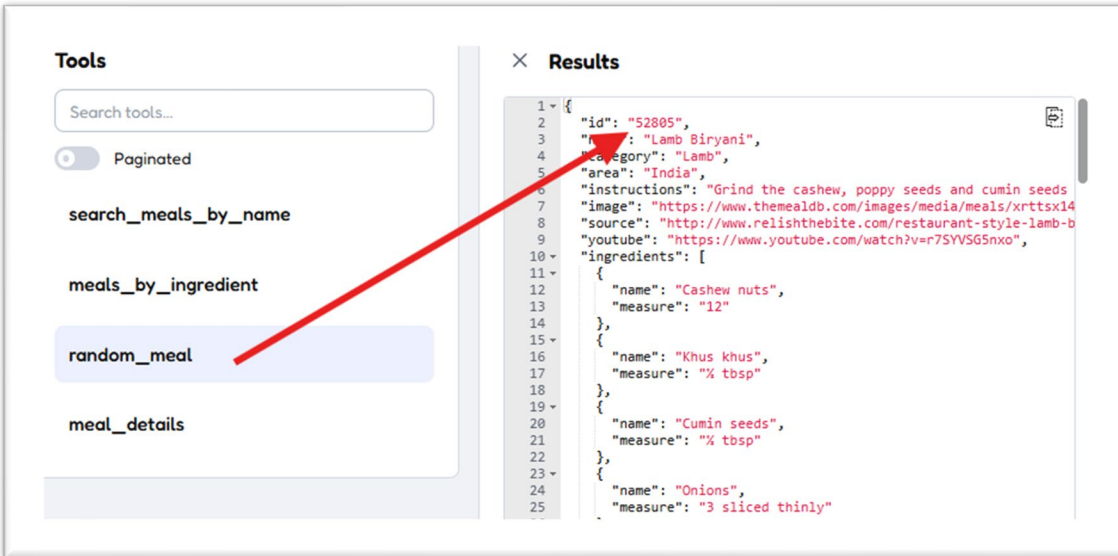
**Ingredient \***

chicken

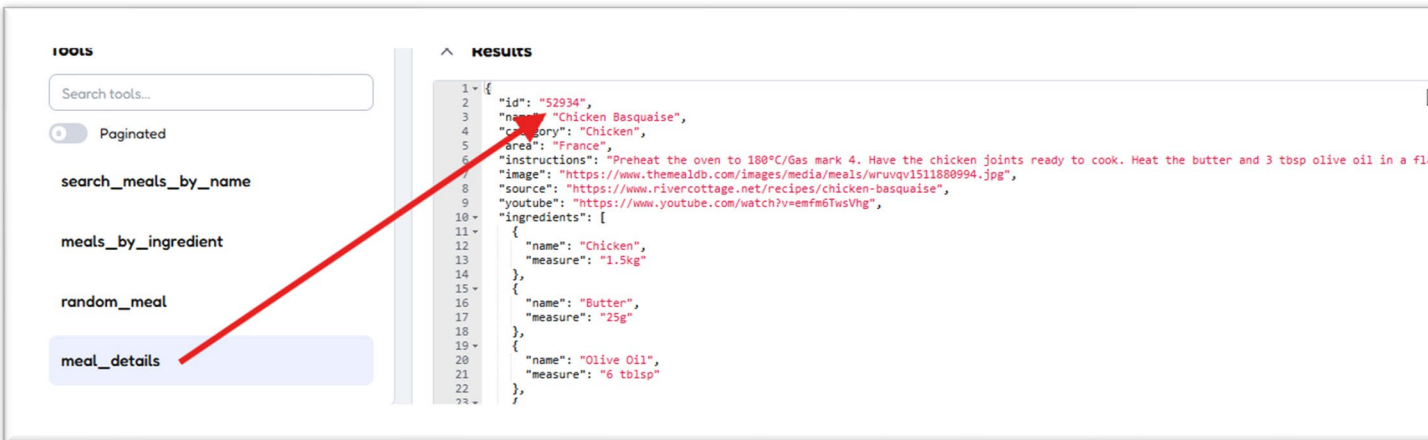
**Limit**

12

**Execute Tool** (button)



Screenshot 2.4 – random\_meal (output)



Screenshot 2.5 – meal\_details (output)

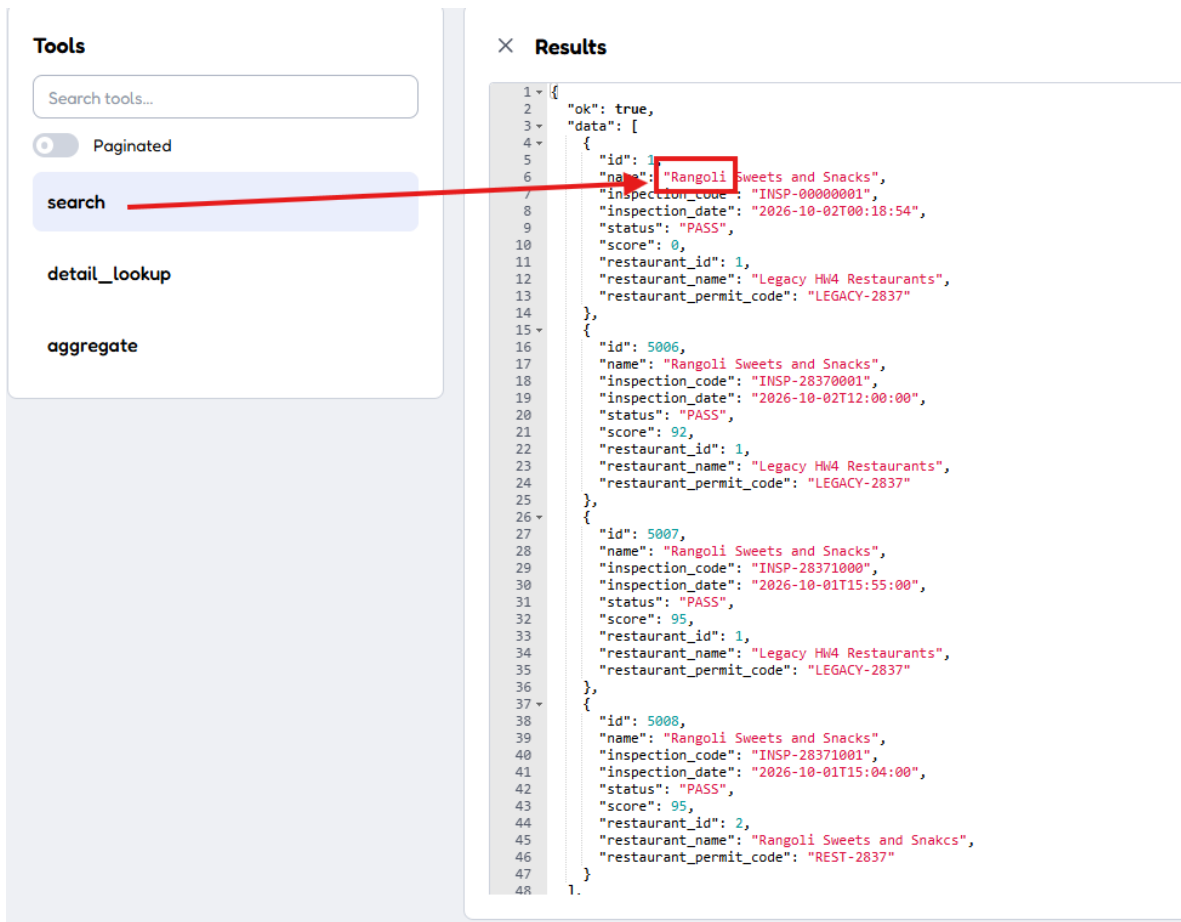
## B. Domain MCP Server (restaurant inspections)

Three tools (search, detail\_lookup, aggregate) share one envelope {ok, data, error}; error is null on success. Logs go to stderr.

Screenshot 2.6 – Inspector: domain server tools



Screenshot 2.7 – search, valid call ("Rangoli")



Screenshot 2.8 – detail\_lookup, valid call (ID 2)

**Tools**

Search tools...

☐ Paginated

**search**

**detail\_lookup**

**aggregate**

**Results**

```
1 {
2   "ok": true,
3   "data": {
4     "id": 2,
5     "name": "Moonlight Garden Restaurant & Banquet Hall",
6     "inspection_code": "INSP-00000002",
7     "inspection_date": "2026-10-02T00:18:00",
8     "status": "WARNING",
9     "score": 60,
10    "restaurant_id": 1,
11    "restaurant_name": "Legacy HW4 Restaurants",
12    "restaurant_permit_code": "LEGACY-2837"
13  },
14  "error": null
15 }
```

**Structured Output**

```
1 {
2   "ok": true,
3   "data": {
4     "id": 2,
5     "name": "Moonlight Garden Restaurant & Banquet Hall",
6     "inspection_code": "INSP-00000002",
7     "inspection_date": "2026-10-02T00:18:00",
8     "status": "WARNING",
9     "score": 60,
10    "restaurant_id": 1,
11    "restaurant_name": "Legacy HW4 Restaurants",
12    "restaurant_permit_code": "LEGACY-2837"
13  },
14  "error": null
15 }
```

Screenshot 2.9 – aggregate, valid call (status = WARNING)

**Tools**

Search tools...

☐ Paginated

**search**

**detail\_lookup**

**aggregate**

**Results**

```
1 {
2   "ok": true,
3   "data": {
4     "status_filter": "WARNING",
5     "total_restaurants": 12,
6     "total_inspections": 1636,
7     "average_score": 0.07,
8     "counts_by_status": {
9       "PASS": 0,
10      "FAIL": 0,
11      "WARNING": 1636
12    }
13  },
14  "error": null
15 }
```

**Structured Output**

```
1 {
2   "ok": true,
3   "data": {
4     "status_filter": "WARNING",
5     "total_restaurants": 12,
6     "total_inspections": 1636,
7     "average_score": 0.07,
8     "counts_by_status": {
9       "PASS": 0,
10      "FAIL": 0,
11      "WARNING": 1636
12    }
13  },
14  "error": null
15 }
```

*Screenshot 2.10 – search, rejected call (empty query)*

The screenshot displays a web application interface with two main panels. The left panel, titled "Tools", contains a search bar with the placeholder text "Search tools...", a "Paginated" toggle switch, and three buttons: "search", "detail\_lookup", and "aggregate". The "search" button is highlighted in blue. The right panel, titled "Results", shows a JSON response in a code editor. The response is a single object with the following structure: 

```
{  "ok": false,  "data": null,  "error": "query cannot be empty"}
```

. Below the code editor, there is a section titled "Structured Output" which displays the same JSON response in a structured format.

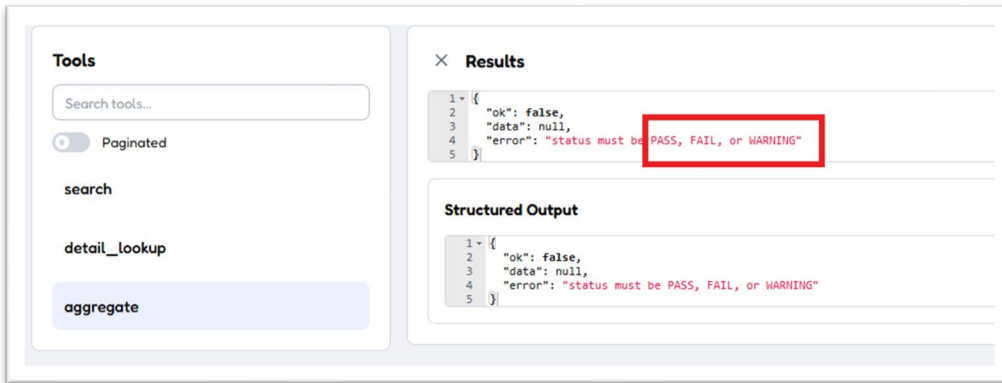
*Screenshot 2.11 – detail\_lookup, rejected call (ID 9999)*

The screenshot displays the same web application interface as Screenshot 2.10, but with the "detail\_lookup" button highlighted in blue. The "Results" panel shows a JSON response in a code editor. The response is a single object with the following structure: 

```
{  "ok": false,  "data": null,  "error": "inspection 9999 was not found"}
```

. The "error" field is highlighted with a red box. Below the code editor, there is a section titled "Structured Output" which displays the same JSON response in a structured format.

Screenshot 2.12 – aggregate, rejected call (status = "testing")



## Part 3 – Tool Contracts Under Stress

### 3.1 Contracts and rejected calls (from Part 2B)

| Tool          | Expected input schema                                | Rejected input                       | Error output                                                                               | Why rejected                               |
|---------------|------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------|
| search        | <code>{"query": string, non-empty}</code>            | <code>{"query": ""}</code>           | <code>{"ok": false, "data": null, "error": "query cannot be empty"}</code>                 | Empty string violates the non-empty rule.  |
| detail_lookup | <code>{"inspection_id": integer}</code>              | <code>{"inspection_id": 9999}</code> | <code>{"ok": false, "data": null, "error": "inspection 9999 was not found"}</code>         | Well-formed ID, but no such record exists. |
| aggregate     | <code>{"status": "PASS"   "FAIL"   "WARNING"}</code> | <code>{"status": "testing"}</code>   | <code>{"ok": false, "data": null, "error": "status must be PASS, FAIL, or WARNING"}</code> | Value is outside the allowed enum.         |

### 3.2 Timeouts and bounded exponential-backoff retry

Retry policy: maximum 3 attempts, with exponential backoff between attempts, and a clean error result after the final failure.

```
{
  "scenario": "success_on_first_attempt",
  "success": true,
  "attempts": 1,
  "latency_ms": 2.586,
  "error": null
},
{
  "scenario": "failure_then_success_after_retry",
  "success": true,
  "attempts": 2,
```

```

"latency_ms": 7.649,
"error": null
},
{
  "scenario": "failure_after_all_allowed_retries",
  "success": false,
  "attempts": 3,
  "latency_ms": 16.347,
  "error": "injected transient failure"
}

```

| Demonstration                     | Success | Attempts | Error                      | Latency(ms)  |
|-----------------------------------|---------|----------|----------------------------|--------------|
| Success on first attempt          | True    | 1        | None                       | <b>2.586</b> |
| Failure, then success after retry | True    | 2        | None                       | <b>7.649</b> |
| Failure after all allowed retries | False   | 3        | injected transient failure | <b>7.649</b> |

### 3.3 Fault-injection results (VERIFY\_SEED = 262837, 50 calls per rate)

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|-----------------------|--------------|-------------------|------------------|
| 0%                    | 100.0%       | 1.743             | 2.242            |
| 20%                   | 98.0%        | 3.897             | 17.706           |
| 50%                   | 92.0%        | 5.819             | 18.786           |

Raw data for all 150 calls: reports/hw05/raw/fault\_injection\_calls.csv; summary: fault\_injection\_summary.json.

### 3.4 Evaluation

Interactive assistant. With 3 attempts, success rose from a raw per-call rate of 80% and 50% to 98% and 92%. For independent failures the expected values are  $1 - 0.2^3 = 99.2\%$  and  $1 - 0.5^3 = 87.5\%$ , so the measured results are consistent with the policy. The extra latency is small (p99 about 18 ms at both failure rates), so most transient failures become invisible to the user, and the remaining failures return a clean error the agent can report. The policy suits an interactive assistant, with one caveat: these latencies come from simulated failures with short backoff. On a real network each failed attempt could cost a full timeout, so the interactive timeout should stay at a few seconds and attempts at three or fewer.

Batch processing. Latency is not user-visible, so I would raise retries to 5 or 6 (at 50% failure,  $1 - 0.5^6 = 98.4\%$  success), lengthen the timeout to 10–30 s, and use exponential backoff with a cap and random jitter (for example 0.5 s doubling up to 30 s) to avoid hammering a recovering service. Items that still fail would be queued for a final retry pass instead of being dropped.

## Part 4 – Safe Tool Entry Point and Offline Tests

`execute_tool(name, inputs)` is the single-entry point for the three domain tools. It reuses the {ok, data, error} envelope, returns a JSON string, and converts every error into an {ok: false} result instead of raising. The tests use assert statements and run without MySQL, MCP Inspector, TheMealDB, Ollama, or internet access.

*Screenshot 4.1 – Offline test runner: PASS/FAIL per test and X/Y summary (7/7 before the Part 5 additions)*

```
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\mcp> python test_execute_tool_offline.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
PASS unknown_tool

7/7 tests passed
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\mcp> █
```

## Part 5 – Agent Loop, Safety Rule, and Reflection

### I. Safety rule

**Rule.** `execute_tool` blocks any `detail_lookup` result whose inspection score is not an integer from 0 to 100. A score outside that range (or a missing score) means the stored data is corrupt, so the tool returns an error instead of passing bad data to the agent, which might repeat it to the user as fact.

```
def enforce_safety_rules(
    tool_name: str,
    data,
) -> str | None:
    """
    Return an error message when a domain safety
    rule is violated.
    """

    if tool_name == "detail_lookup":
        if isinstance(data, dict):
            score = data.get("score")

            if (
                not isinstance(score, int)
                or score < 0
                or score > 100
            ):
                return (
```

```

        "Safety rule blocked the response: "
        "inspection score must be between "
        "0 and 100"
    )

    return None

```

*Screenshot 5.1 – Offline tests including safety\_rule\_allows\_valid\_score and safety\_rule\_blocks\_invalid\_score (9/9)*

```

PS C:\SJSURRepo\Data260\Data260-2837\code\web_application\mcp>
PS C:\SJSURRepo\Data260\Data260-2837\code\web_application\mcp> python test_execute_tool_offline.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
PASS unknown_tool
PASS safety_rule_allows_valid_score
PASS safety_rule_blocks_invalid_score

9/9 tests passed
PS C:\SJSURRepo\Data260\Data260-2837\code\web_application\mcp>

```

## II. Agent loop

run\_agent(user\_input) uses the local Ollama model and the three domain tools through execute\_tool. It keeps a turn counter, enforces max\_steps, and logs each step, tool call, input, result, and final stop reason to a JSONL file.

*Screenshot 5.2 – Single-request agent run (python agent.py)*

```

python agent.py

{"answer": "The inspections for Rangoli are as follows:\n\n1. Rangoli Sweets and Snacks, Restaura",
 "steps": 2,
 "tool_calls": 1,
 "stop_reason": "normal_completion"}

PS C:\SJSURRepo\Data260\Data260-2837\code\web_application\mcp>

```

```
python agent.py
```

```
{
```

```
"answer": "The inspections for Rangoli are as follows:\n\n1. Rangoli Sweets and Snacks, Restaurant Name: Legacy HW4 Restaurants, Inspection Date: 2026-10-02, Score: 0, Status: PASS.\n\n2. Rangoli Sweets and Snacks, Restaurant Name: Legacy HW4 Restaurants, Inspection Date: 2026-10-02, Score: 92, Status: PASS.\n\n3. Rangoli Sweets and Snacks, Restaurant Name: Legacy HW4 Restaurants, Inspection Date: 2026-10-01, Score: 95, Status: PASS.\n\n4. Rangoli Sweets and Snacks, Restaurant Name: Rangoli Sweets and Snacks, Inspection Date: 2026-10-01, Score: 95, Status: PASS.",
"steps": 2,
"tool_calls": 1,
"stop_reason": "normal_completion"
}
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\mcp>
```

### III. Additional tests

*Screenshot 5.3 – Final offline suite: 10/10 including agent\_stops\_at\_max\_steps (MockModel)*

```
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\mcp> python test_execute_tool_offline.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
PASS unknown_tool
PASS safety_rule_allows_valid_score
PASS safety_rule_blocks_invalid_score
PASS agent_stops_at_max_steps
10/10 tests passed
PS C:\SJSURepo\Data260\Data260-2837\code\web_application\mcp>
```

## IV. Metrics

### Screenshot 5.4 – Four agent scenarios (python run\_agent\_scenarios.py)

```
PS C:\SJSURRepo\Data260\Data260-2837\code\web_application\mcp> python .\run_agent_scenarios.py

Running scenario: search_rangoli
{
  "scenario": "search_rangoli",
  "input": "Find all inspections for Rangoli and summarize their scores and statuses.",
  "steps": 4,
  "stop_reason": "model_error",
  "tool_calls": 3,
  "answer": null
}

Running scenario: inspection_detail
{
  "scenario": "inspection_detail",
  "input": "Give me the details for inspection 5006, including its status and score.",
  "steps": 4,
  "stop_reason": "normal_completion",
  "tool_calls": 3,
  "answer": "The inspection ID 5006 for the restaurant 'Legacy HW4 Restaurants' is categorized as a PASS with a score of 92."
}

Running scenario: pass_aggregate
{
  "scenario": "pass_aggregate",
  "input": "How many PASS inspections are there and what is the average score?",
  "steps": 4,
  "stop_reason": "normal_completion",
  "tool_calls": 3,
  "answer": "There are 1713 PASS inspections with an average score of 0.16."
}
```

| Scenario          | Steps | Tool calls | Stop reason       | Observation                                                                                  |
|-------------------|-------|------------|-------------------|----------------------------------------------------------------------------------------------|
| search_rangoli    | 4     | 3          | model_error       | Model failed at step 4; no final answer (answer = null).                                     |
| inspection_detail | 4     | 3          | normal_completion | Correct answer (ID 5006, PASS, 92) but used 3 tool calls for one lookup.                     |
| pass_aggregate    | 4     | 3          | normal_completion | Completed, but the answer (1713 PASS, average 0.16) reflects mostly zero scores in the data. |
| fail_search       | 6     | 5          | normal_completion | Confused answer: model passed a non-integer inspection_id and the tool rejected it.          |

## V. Reflection

For this reflection I used the single-request run from Screenshot 5.2, where I ran python agent.py with a request to find the inspections for Rangoli. The output shows 2 steps, 1 tool call, and a stop reason of normal\_completion.

When the request came in, the harness started the agent loop with a maximum step limit. In step 1, the local Ollama model chose the search tool instead of answering from its own knowledge. The model never touches the database directly. Every tool request goes through the single execute\_tool function, which validates the input, runs the tool, and returns the result in the {ok, data, error} format. The harness logged the tool name, the input, and the result to the JSONL log.

In step 2, the model read the returned records and wrote its final answer. It listed four Rangoli inspections with their restaurant names, dates, scores (0, 92, 95, 95), and PASS statuses.

Because the model answered in text and asked for no more tools, the harness treated that as the final answer and stopped. The stop reason was `normal_completion`, which means the agent finished before reaching the step limit.

The run also showed me two data issues. The first inspection has a score of 0 but a PASS status, which looks like placeholder data from my HW4 migration. One record also has the misspelling "Snakcs", which the model copied from the database. So `normal_completion` only means the loop ended cleanly, not that every value in the answer is correct.

The design worked as intended: the model chooses an action, `execute_tool` runs it safely, the result goes back to the model, and the loop stops with a logged reason.

## VI. Verification

*Screenshot 5.5 – verify\_hw05.py: 10/10 checks, verification.json written to reports/hw05/*

```
C:\SJSURRepo\Data260\Data260-2837\code\web_application>python verify_hw05.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
PASS unknown_tool
PASS safety_rule_allows_valid_score
PASS safety_rule_blocks_invalid_score
PASS agent_stops_at_max_steps

10/10 tests passed

HW5 verification completed.
Output written to: C:\SJSURRepo\Data260\Data260-2837\reports\hw05\verification.json
10/10 checks passed

C:\SJSURRepo\Data260\Data260-2837\code\web_application>|
```

## AI Use

### 1. What I used an AI assistant for and what I did myself:

I used AI for code validation and issues resolution and also drafting the final report. I used an AI assistant to help understand the HW5 requirements, plan the software architecture, extend the existing HW4

I did all the execution, and you can see my “last 4 digit of student ID (018322837) i.e. 2837 in every output and execution.

I used my own existing HW4 repository, database, credentials, local environment, and assigned domain data. I manually created the files in my repository, executed the commands, tested the application, reviewed the outputs, and captured the required evidence screenshots.

**2. One AI-produced output that was wrong or unsuitable:** My verification result was getting wrong specifically for “backend server” running health check. And AI was giving all kinds of explanation that why your server is not running etc.

**3. How I detected it:** I reviewed my outputs and everything looked good to me. Moreover, all my earlier tests and processes were working fine, hence I was sure my code/applications were working fine. I once again reviewed my `verification.py` code and realized that something is not right.

**4. What I changed and why it works now:** I looked closely at my “`check_backend_running`” method and there was a typo in my `status_code`. Instead of 200 it was “280”. Once I fixed it, all my validation passed.